



UNIDADE IV

Projeto de Sistemas Orientado a Objetos

Prof. Me. Edson Moreno

U IV – Refinando a modelagem de aspectos dinâmicos do *software*

- Até o momento, nas unidades I, II e III foram vistas a passagem do modelo de análise para o modelo de projeto, o que deu entrada na fase de projetos, nas atividades de projeto de dados e classes e também nas atividades de projeto arquitetural.
- Agora já se sabe quais os objetivos de cada atividade e quais modelos precisam ser produzidos na componentização do sistema, com o auxílio da UML.
- Na unidade IV será visto como produzir os modelos que tratam do projeto de interfaces e o projeto de componentes com objetivo de prover maior qualidade da informação que é passada para a equipe de construção e demais envolvidos no projeto.
- Neste contexto, os principais diagramas a serem abordados na unidade IV são: diagrama de comunicação, diagrama de máquina de estados e o diagrama de pacotes.



Diagrama de comunicação

- O diagrama de comunicação passou a ter esse nome a partir da versão 2.0 da UML; nas versões anteriores, esse diagrama é chamado de diagrama de colaboração (UML-DIAGRAMS, 2009-2014b).
- A característica marcante do diagrama de comunicação é a forte semelhança com o diagrama de sequência. As informações modeladas em ambos são, no geral, as mesmas, todavia, a representação em cada um dos modelos possui ênfases diferentes.
- O diagrama de comunicação é um tipo de diagrama comportamental da UML que representa as interações de dois objetos e suas partes utilizando para isso uma sequência de mensagens representadas de forma livre de formatação (UML-DIAGRAMS, 2009-2014b).
- O diagrama de comunicação é um complemento do diagrama de sequência.

Saiba mais:
UML-DIAGRAMS. Timing diagrams, 2009-2014a

Fonte:
<http://www.uml-diagrams.org/timing-diagrams.html>.

Diagrama de comunicação – Características

- Enquanto o diagrama de sequência dá ênfase à troca de mensagens em uma linha de tempo, o diagrama de comunicação dá ênfase a como os objetos estão interligados e quais mensagens são trocadas entre eles para realizar uma determinada tarefa.
- No diagrama de comunicação as mensagens possuem uma numeração, é como se elas fossem etiquetadas com uma numeração em ordem crescente, e é essa sequência numérica que representa a sequência em que as mensagens são trocadas entre os objetos.
- O início das mensagens é identificado por 1.0 seguindo essa mensagem do objeto que envia até o objeto que a recebe, e assim sucessivamente.
- Os diagramas de sequência e de comunicação são isomórficos, ou seja, um pode ser transformado no outro.



Saiba mais:

IBM Knowledge Center. Criando Diagramas de Comunicação.

Fonte:

https://www.ibm.com/support/knowledgecenter/pt-br/SS8PJ7_9.6.1/com.ibm.xtools.sequence.doc/topics/twrkcommnd.html.

Diagrama de comunicação – Quando usar!

- Se o seu objetivo for representar a interação dos objetos no decorrer de uma linha do tempo, então você deverá usar:
- Se o seu objetivo for dar ênfase à interação desses objetos no contexto do sistema, então você deverá usar:

Diagrama de
sequência

Diagrama de
comunicação

Lembrete:

Diagramas de sequência e de comunicação são complementares.

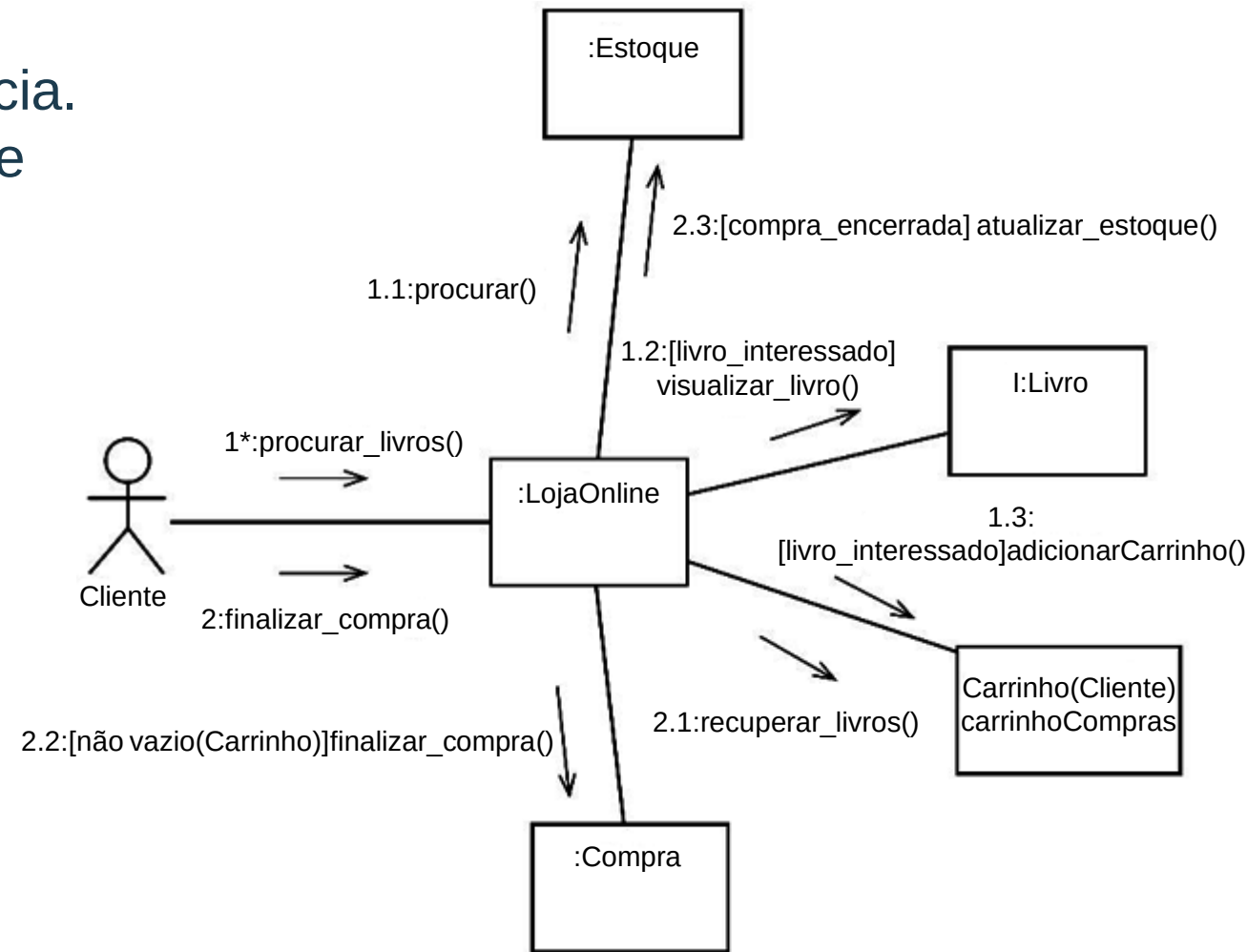
O desenvolvimento do diagrama de comunicação a partir de um de sequência (ou vice-versa) é também uma atividade revisional e de melhoria, que pode:

Identificar pontos de falhas ou omissões no projeto.

Erros de relacionamento no projeto arquitetural.

Diagrama de comunicação – Elementos que o compõem

- Objetos: têm a mesma conotação do objeto que é representado no diagrama de sequência.
- Vínculos: identificam uma ligação de troca de mensagens entre dois objetos.
- Mensagens: análogas às mensagens do diagrama de sequência, exceto que são identificadas numericamente.
- Atores: os atores representados são os mesmos do diagrama de sequência e do diagrama de casos de uso.



Fonte: VERSOLATTO (2015).

Diagrama de comunicação – Composição

A	Ator – o mesmo do caso de uso
B	Objeto – símbolo retângulo
C	Representação de vínculo
D	Indicador: sequência de mensagens
E	Mensagem
F	Parâmetro de mensagem
G	Declaração do objeto (nome e tipo)
H	Descrição do objeto (seletor e tipo)

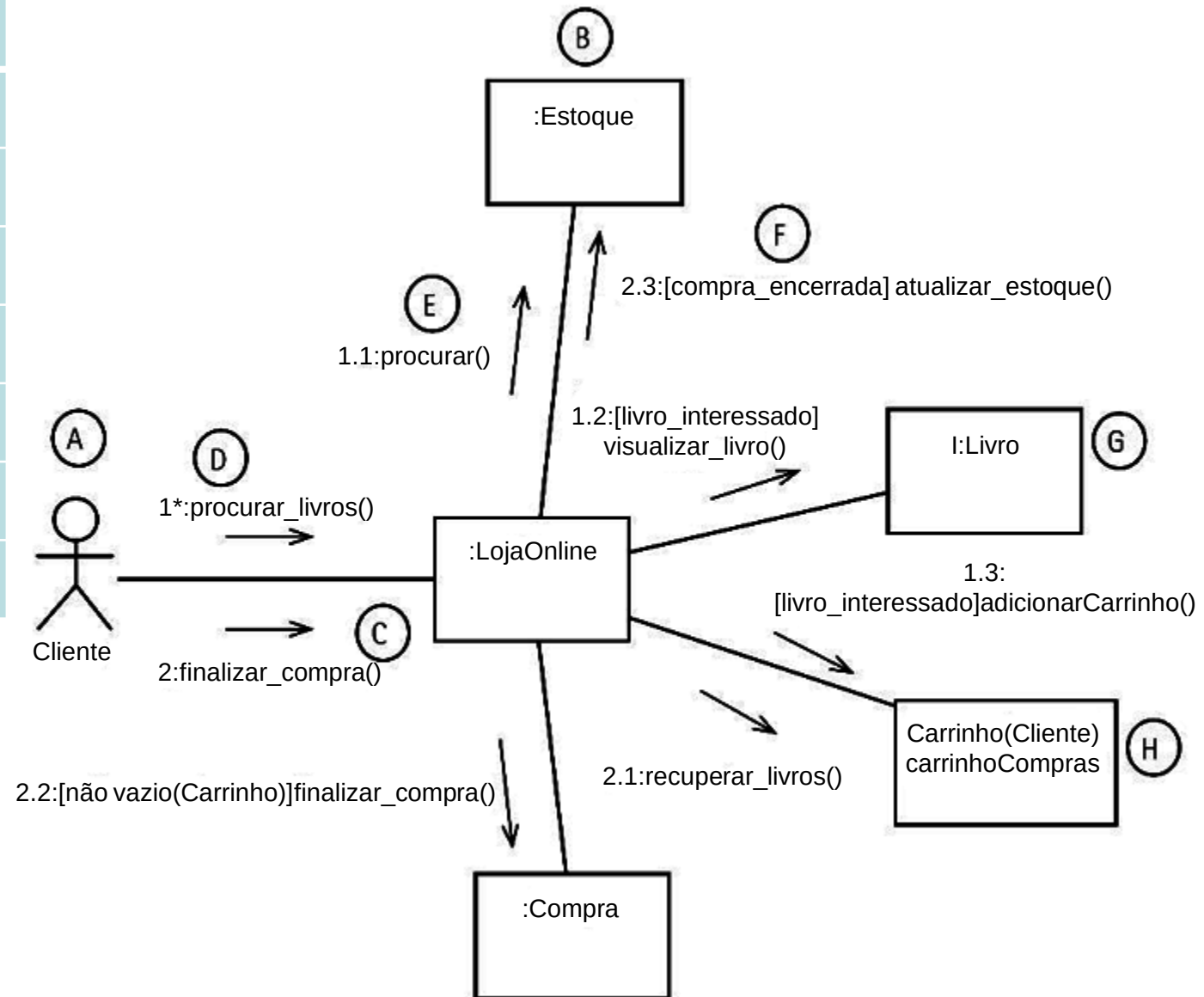


Diagrama de comunicação – Diagrama de sequência equivalente

Observe:

- A não ocorrência da linha de retorno de mensagem no diagrama de comunicação.
- As mensagens procurar_livros() e finalizarCompra() iniciam a partir do Cliente. Logo, a numeração no diagrama de colaboração é de nível um, ou seja: 1:procurar_livros() e 2:finalizarCompra().

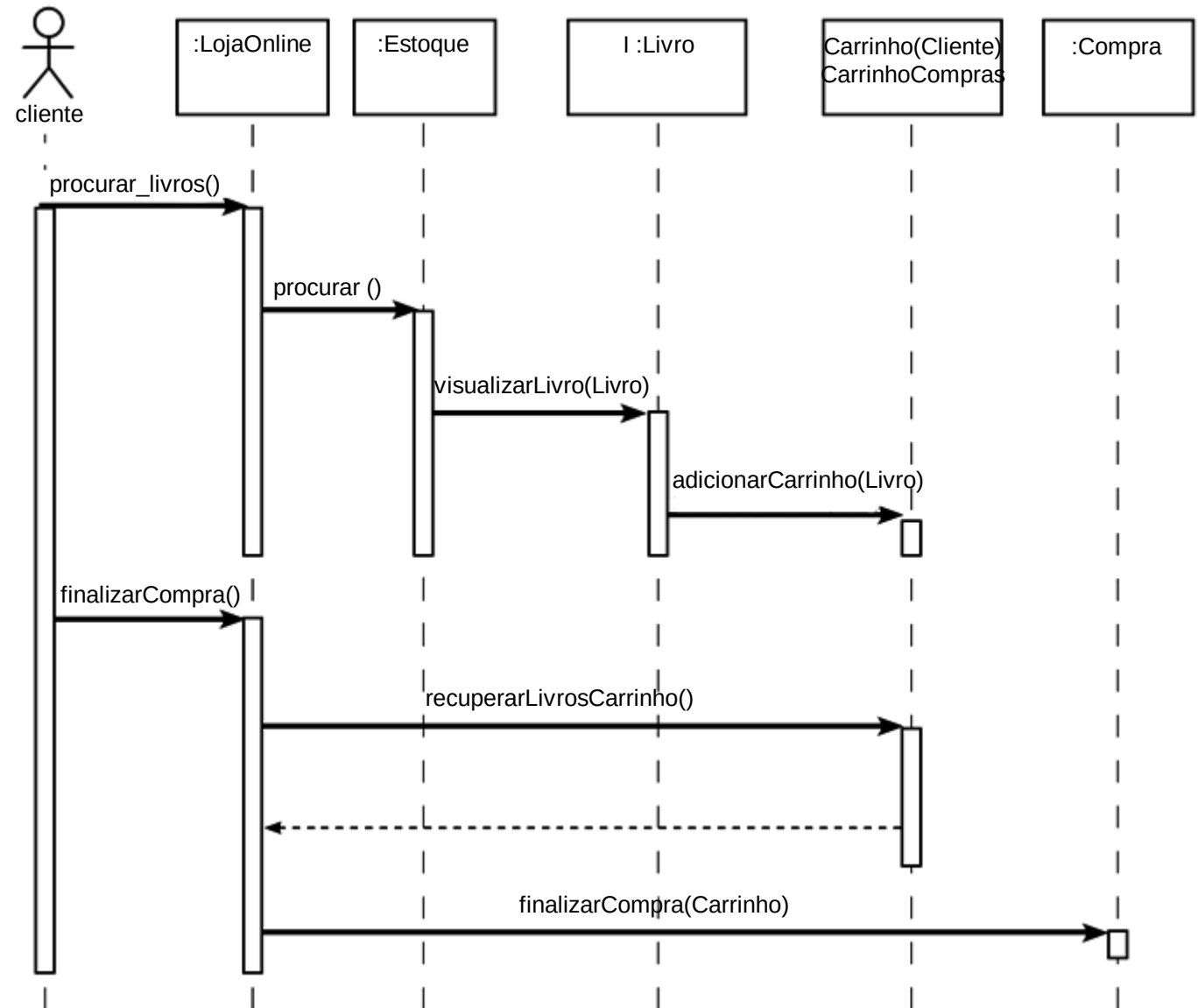
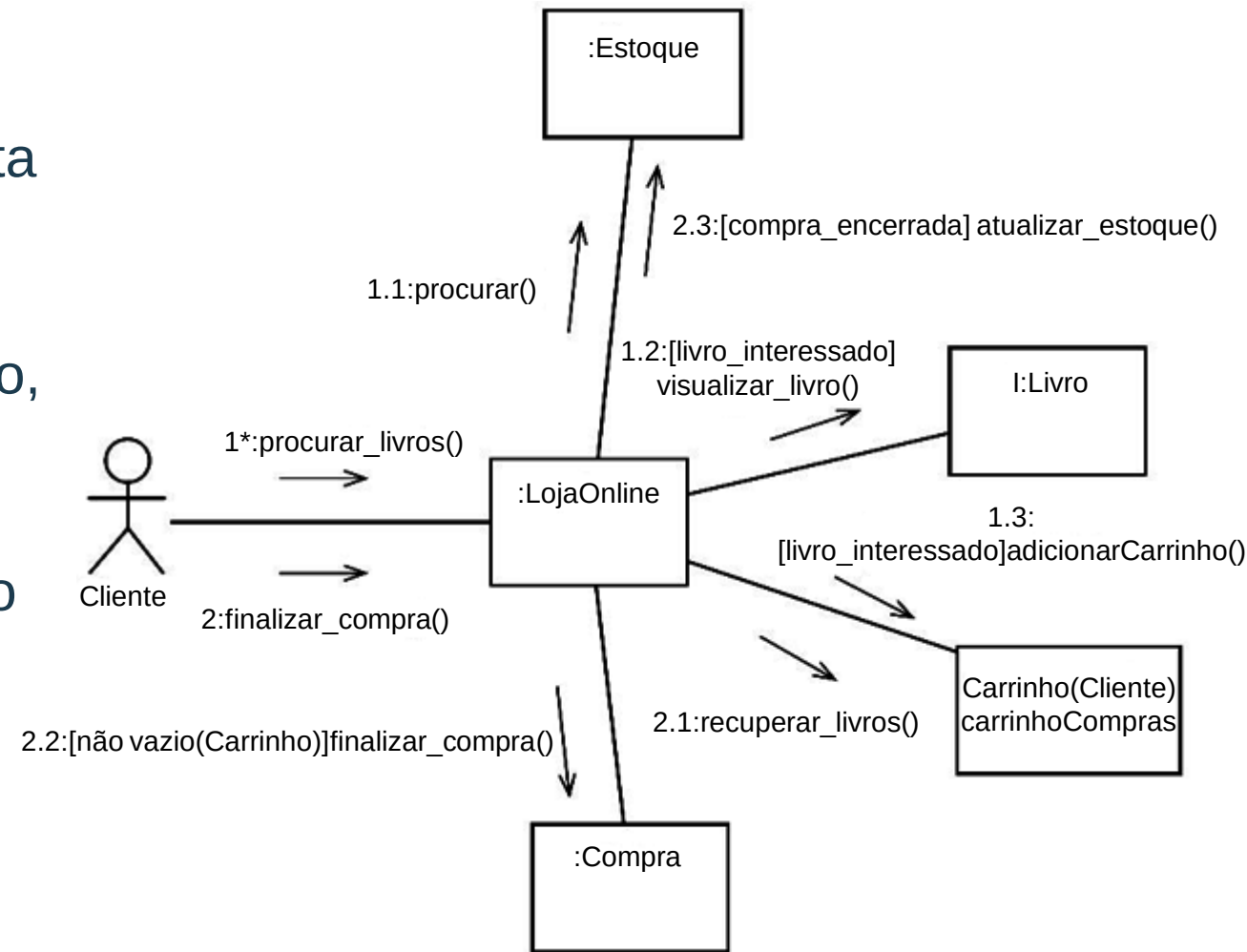


Diagrama de comunicação – Analogia com diagrama de sequência

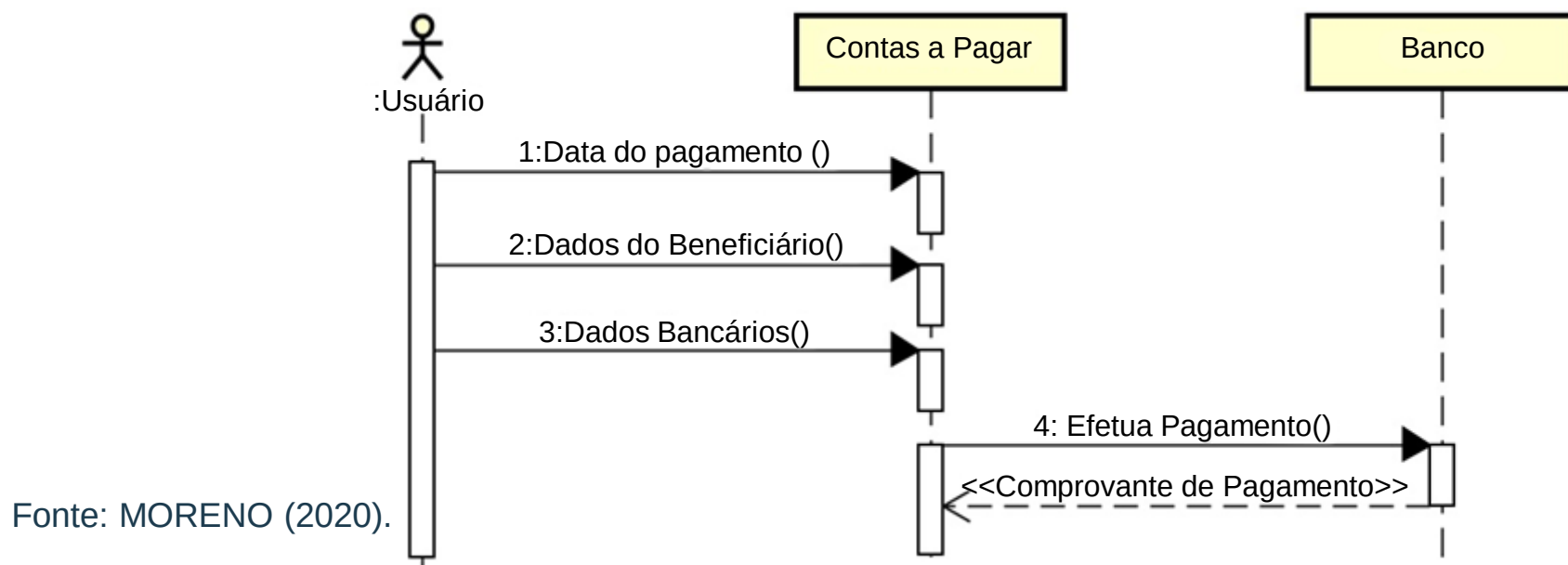
- A representação desse cenário utilizando o diagrama de comunicação projeta um diagrama que, em linhas gerais, se apresenta mais fácil de ser entendido.
- Nas iterações de projeto, em uma rápida abordagem com o diagrama de comunicação, pode se ter uma modelagem de cenários mais simples.
- Não esquecer que sempre será necessário o diagrama de sequência para a codificação.



Fonte: VERSOLATTO (2015).

Interatividade

- Observe o diagrama de sequência abaixo, sobre Contas a Pagar. Assinale a alternativa que apresenta a melhor interpretação deste diagrama para a montagem do diagrama de comunicação.
- a) A numeração de Dados do Beneficiário() é 1.1.
- b) A numeração de Efetua Pagamento () é 3.1.
- c) O ator é Usuário; e Contas a Pagar e Banco são estados da comunicação.
- d) O estereótipo de retorno <<Comprovante de Pagamento >> recebe a numeração 4.1.
- e) O indicador de Efetua Pagamento() por estar no nível dois é uma linha tracejada.



Resposta

- Observe o diagrama de sequência abaixo, sobre Contas a Pagar. Assinale a alternativa que apresenta a melhor interpretação deste diagrama para a montagem do diagrama de comunicação.
- a) A numeração de Dados do Beneficiário() é 1.1.
- b) A numeração de Efetua Pagamento () é 3.1.**
- c) O ator é Usuário; e Contas a Pagar e Banco são estados da comunicação.
- d) O estereótipo de retorno <<Comprovante de Pagamento >> recebe a numeração 4.1.
- e) O indicador de Efetua Pagamento() por estar no nível dois é uma linha tracejada.

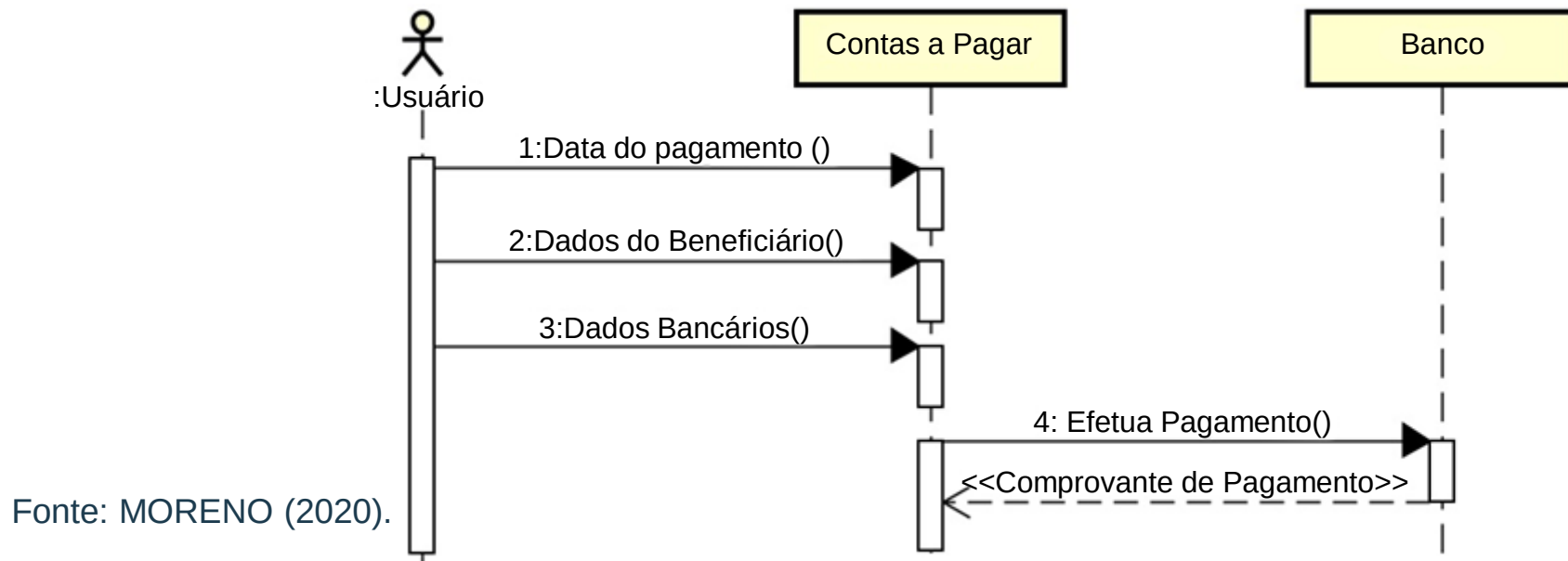


Diagrama de máquina de estado

- O Diagrama de Estados era conhecido nas versões anteriores da linguagem como Diagrama de Gráfico de Estados, tendo assumido a nova nomenclatura a partir da versão 2.0.
- O Diagrama de Estados procura acompanhar as mudanças sofridas por um objeto dentro de um determinado processo.
- O elemento estado a ser representado no diagrama de estados é, na maioria das vezes, uma instância de uma classe, um objeto, uma vez que um objeto, pelos princípios da orientação a objetos, possui um estado (LARMAN, 2007).
- O diagrama de estado pode representar também a mudança de estados de um caso de uso.

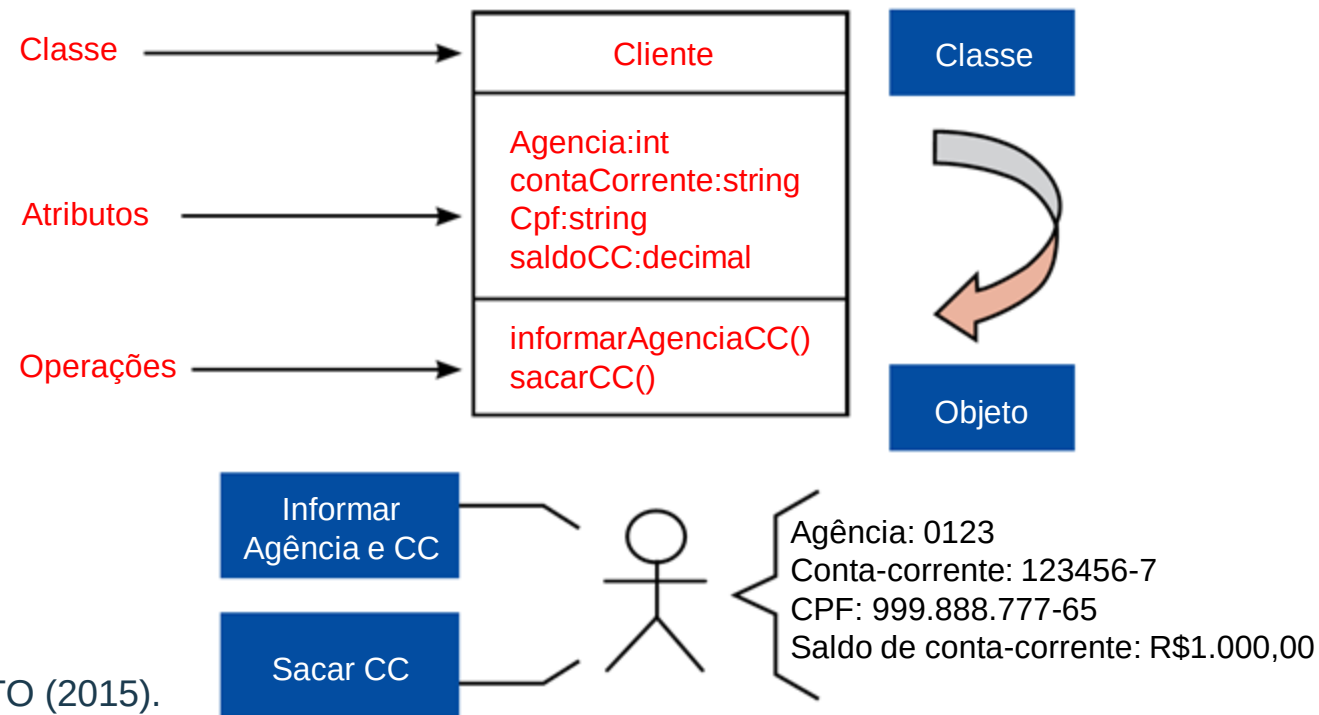
Saiba mais:

IBM Knowledge Center. Modelando o Comportamento do Objeto Utilizando Diagramas de Máquina de Estado.

Fonte: https://www.ibm.com/support/knowledgecenter/pt-br/SS4JE2_7.5.5/com.ibm.xtools.modeler.doc/topics/twrksmd.html. Consultado em 25/06/2020.

Diagrama de máquina de estado – Entendendo o estado

- A partir do momento em que essa classe se torna um objeto no sistema, os atributos passam a ter um determinado valor.
- Esse objeto passa a ter uma identidade única dentro do sistema, passa a estar apto a receber e enviar mensagens, ou seja, ele passa a ser concretamente um cliente.



Fonte: VERSOLATTO (2015).

- No momento, o cliente possui R\$ 1.000,00 em sua conta-corrente. Podemos dizer então que, nesse momento, ele possui este estado.
- Pouco tempo depois, o cliente efetua um saque de R\$ 300,00, seu estado será alterado, pois o valor do atributo “saldoCC” será alterado, ou seja, haverá uma mudança de estado.

Diagrama de máquina de estado – Aplicação

- A figura representa o diagrama de estado para um terminal bancário.

Elementos básicos do diagrama de estados	
Símbolo	Significado
Estado	Representa uma determinada situação de um elemento em um determinado momento.
→	Transição do estado de um evento.
●	Estado inicial
⦿	Estado final

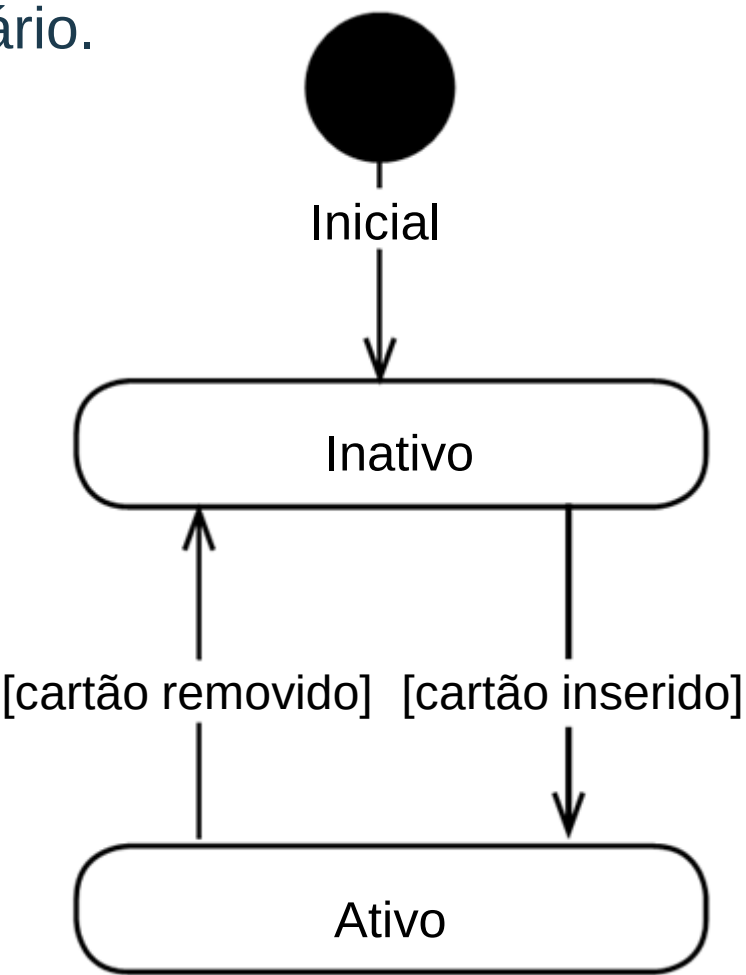
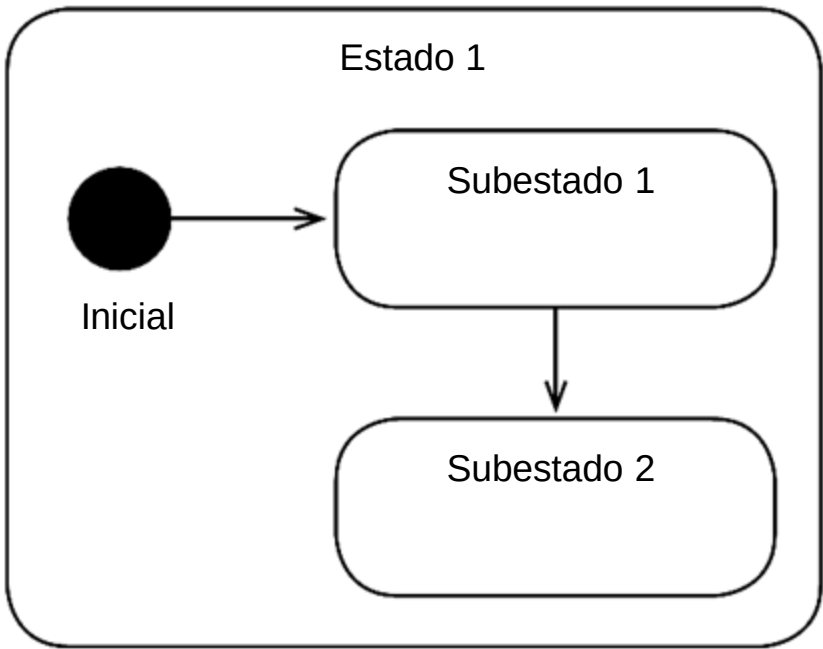


Diagrama de máquina de estado – Classificação

O estado de um objeto pode ser classificado como:

- Simples: quando é autossuficiente, ou seja, não possui a necessidade da composição ou da divisão em estados menores.
- Composto: quando um estado contém internamente dois ou mais estados, chamados subestados, como mostra o exemplo da figura.

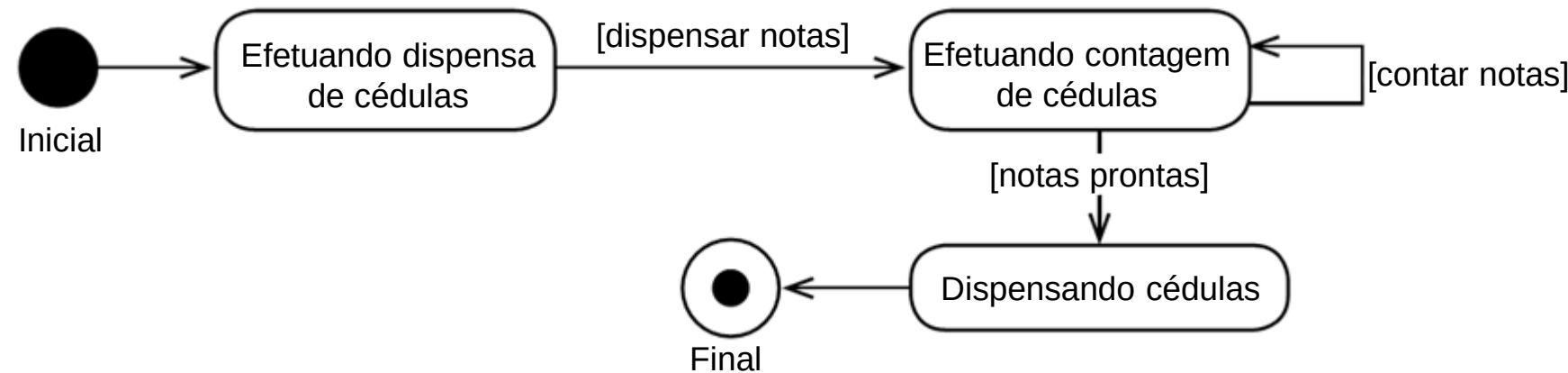


Fonte: VERSOLATTO (2015).

Atividades que os estados assumem	
Atividade: entrada (<i>entry</i>)	Primeira atividade a ser executada quando um objeto assume um determinado estado.
Atividade: fazer (<i>do</i>):	Atividades executadas quando um objeto se encontra em um determinado estado
Atividades de saída (<i>exit</i>):	Atividades executadas no momento em que um objeto sai de um determinado estado.

Diagrama de máquina de estado – Transição

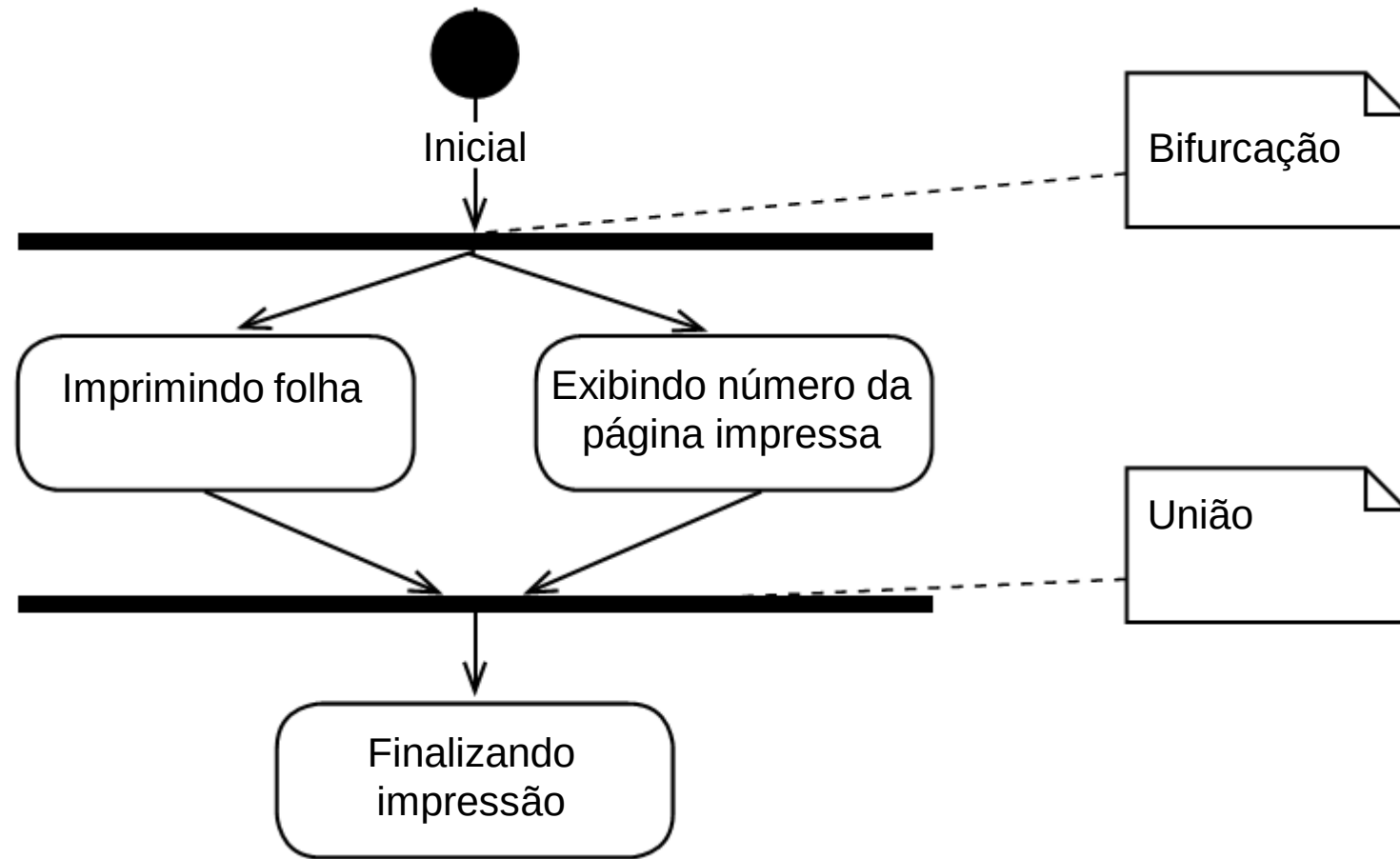
- As transições não produzem alteração no estado de um objeto. Ocorrem durante o estado do objeto, sem que esse estado seja modificado. As transições são chamadas de:
- Transições internas: são transições que acontecem enquanto um objeto se encontra em um determinado estado e que não promovem modificação neste estado do objeto. Essas transições executam atividades de fazer (do).
- Autotransições: são transições que executam atividades do tipo sair (exit) de um objeto, executam uma determinada ação e voltam para o estado de que saíram sem que haja alteração desse estado, como mostra a figura:



Fonte: VERSOLATTO (2015).

Diagrama de máquina de estado – Paralelismo

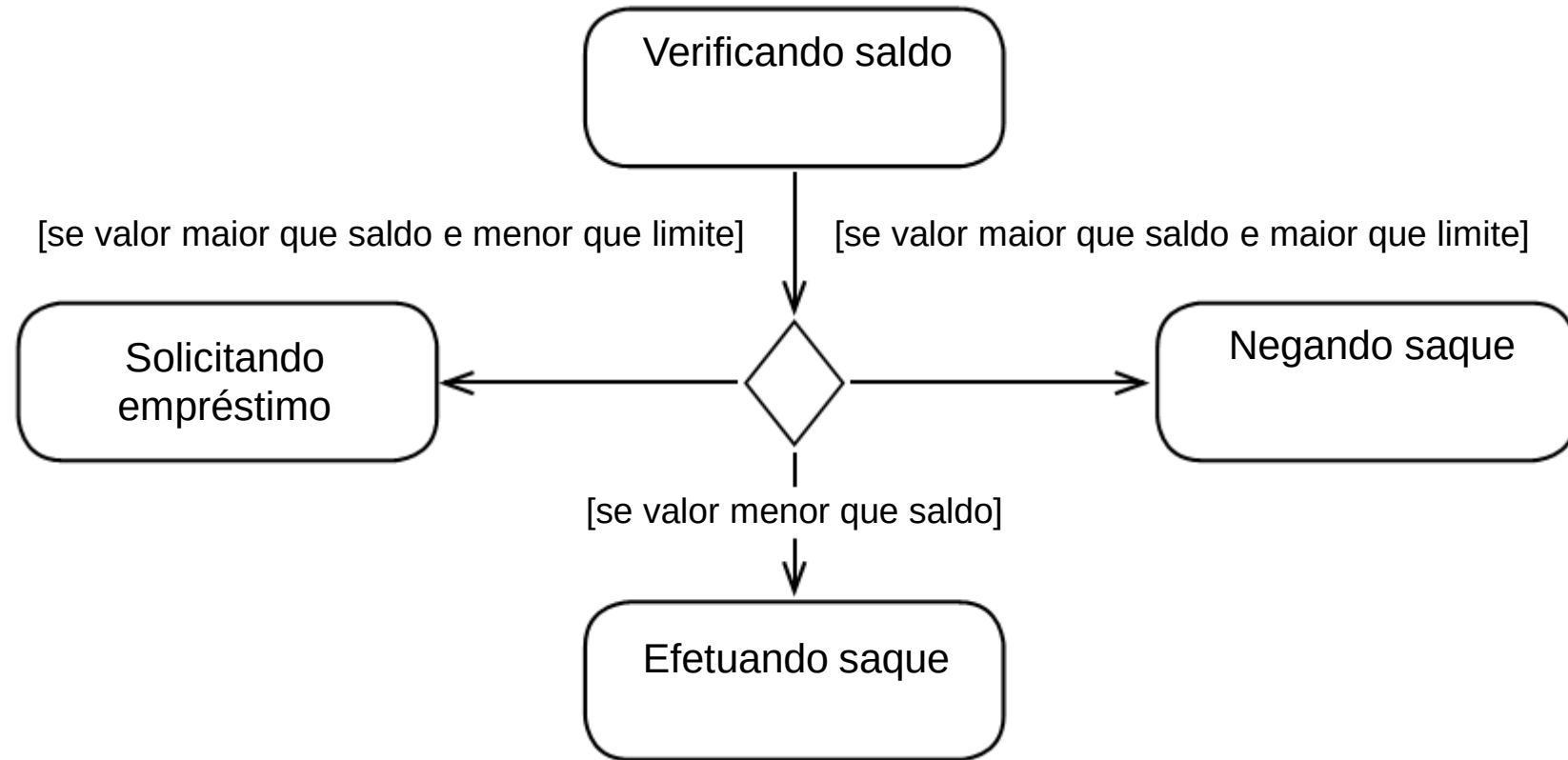
- No diagrama de máquina de estados é possível representar processamentos paralelos.
- Para representar esse paralelismo são utilizados alguns mecanismos, como a barra de bifurcação e a barra de união, como mostra o figura:



Fonte: VERSOLATTO (2015).

Diagrama de máquina de estado – Pseudoestado de escolha

- O pseudoestado de escolha representa um elemento de ponto de decisão.
- Exemplo: comando do tipo *if/else*.



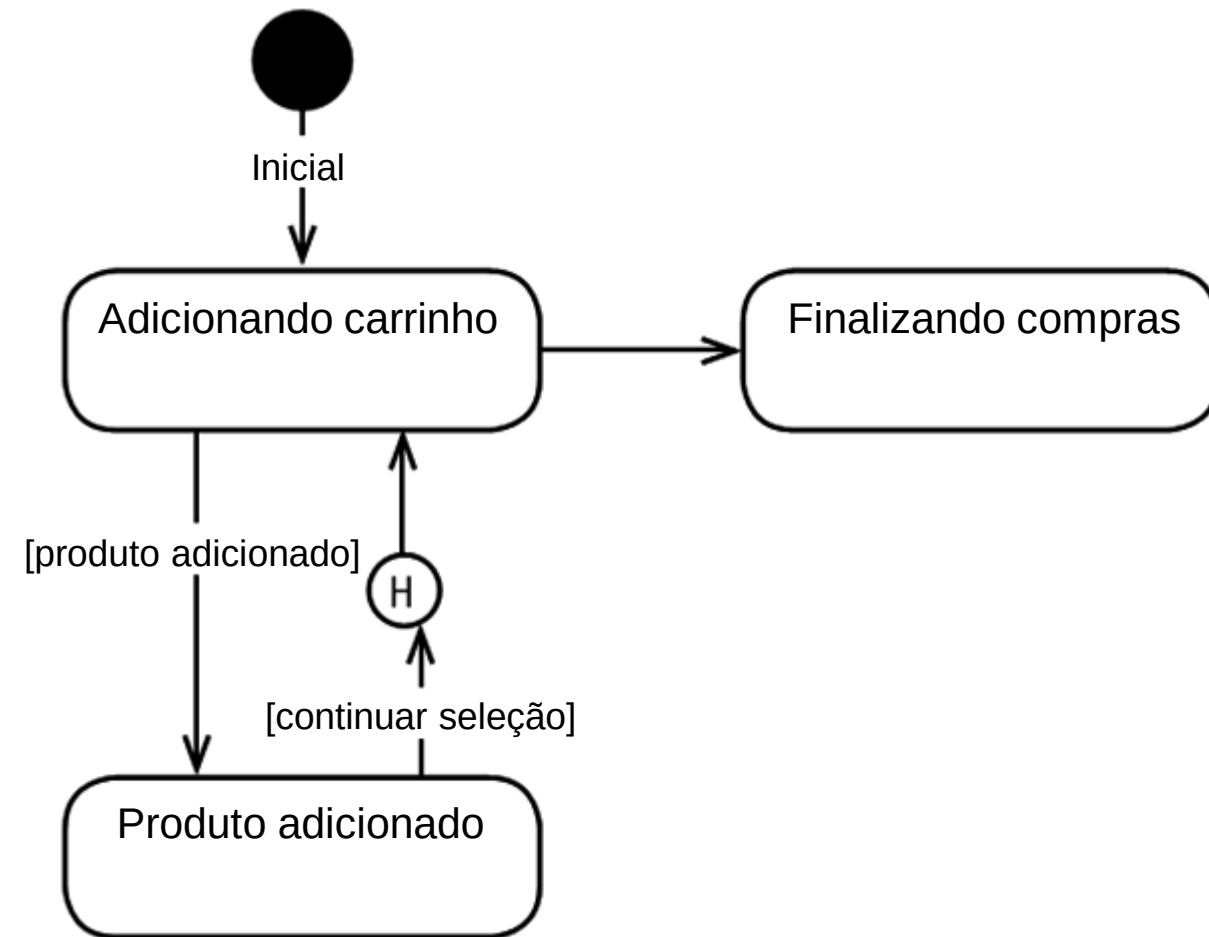
Fonte: VERSOLATTO (2015).

Análise do diagrama de estados da figura:

O estado "Verificando saldo" verifica a movimentação do cliente. Se valor maior que saldo (verifica limite) e se valor menor que saldo permite saque.

Diagrama de máquina de estado – Pseudoestado de história

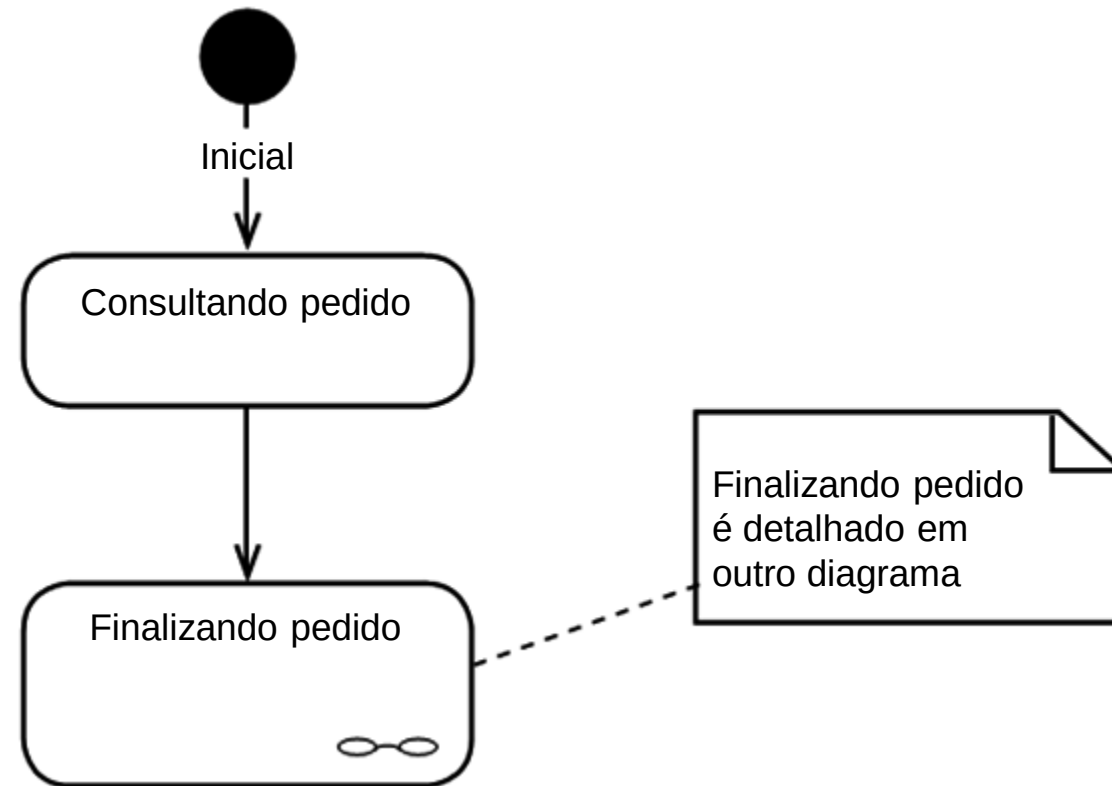
- O pseudoestado de história é utilizado para representar o registro do último estado em que um objeto se encontrava quando, por alguma razão, o processo foi interrompido.
- O pseudoestado de história é usado para quando o processo precisar voltar exatamente no ponto em que o estado se encontrava antes de uma interrupção.
- O símbolo (H) é usado para representar esse pseudoestado.



Fonte: VERSOLATTO (2015).

Diagrama de máquina de estado – Pseudoestado de submáquina

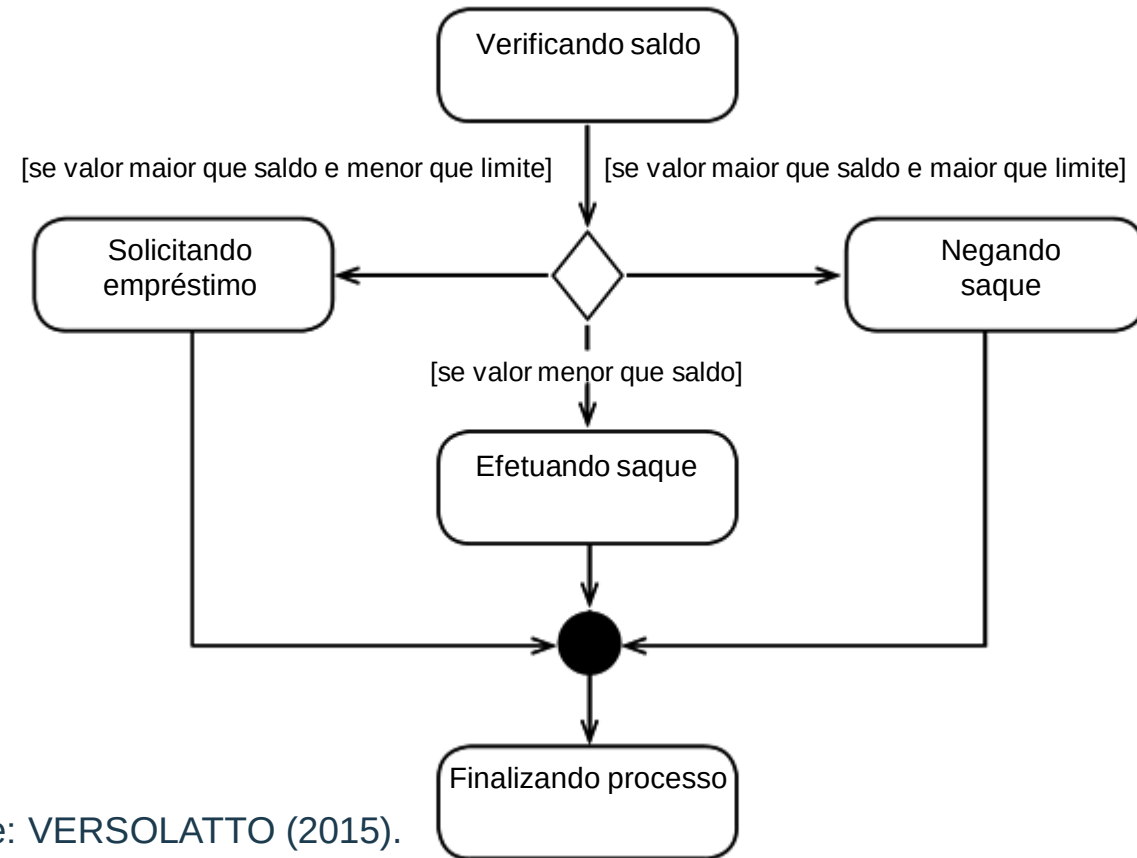
- O pseudoestado de submáquina utiliza um estado de submáquina quando precisamos representar um estado composto de alta complexidade.
- Um estado composto pode representar outros processos.
- Para entender como funciona um estado composto é necessário consultar seu projeto.
- O pseudoestado de submáquina é representado por  .



Fonte: VERSOLATTO (2015).

Diagrama de máquina de estado – Pseudoestado de junção

- O pseudoestado de junção é utilizado para representar a união de múltiplos fluxos em um único ponto.
- O pseudoestado de junção é representado pelo símbolo ●.



Fonte: VERSOLATTO (2015).

Análise do diagrama de estados da figura:

Para que ocorra “Finalizando processo” é necessário que a combinação dos estados gere pelo menos um resultado que reflete a situação da operação.

Interatividade

Analise cada definição como Verdadeira (V) ou Falsa (F) e assinale a alternativa correta.

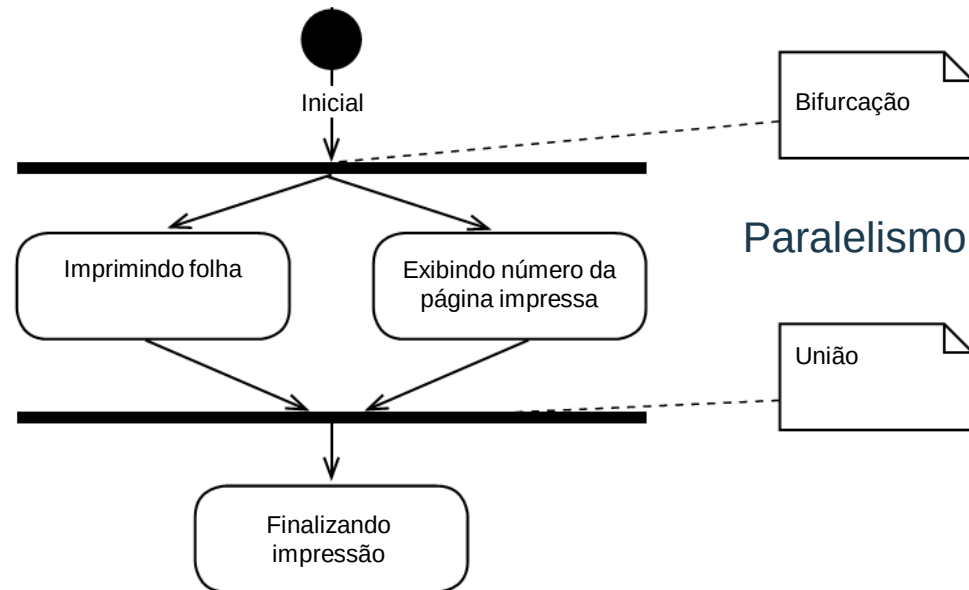
- I. O estado representa a situação de um determinado elemento em um determinado momento.
 - II. É paralelismo quando um estado envia uma ordem simultânea para dois outros estados.
 - III. Um “relatório de log” é considerado um pseudoestado de história.
- a) F, F, F.
 - b) F, V, F.
 - c) V, F, V.
 - d) V, V, F.
 - e) V, V, V.

Resposta

Analise cada definição como Verdadeira (V) ou Falsa (F) e assinale a alternativa correta.

- I. O estado representa a situação de um determinado elemento em um determinado momento.
- II. É paralelismo quando um estado envia uma ordem simultânea para dois outros estados.
- III. Um “relatório de log” é considerado um pseudoestado de história.

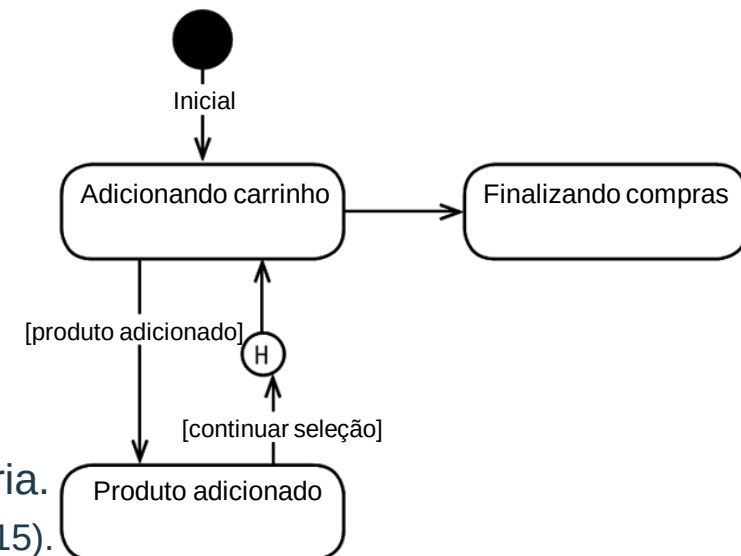
- a) F, F, F.
- b) F, V, F.
- c) V, F, V.
- d) **V, V, F.**
- e) V, V, V.



Fonte: VERSOLATTO (2015).

Pseudoestado de história.

Fonte: VERSOLATTO (2015).

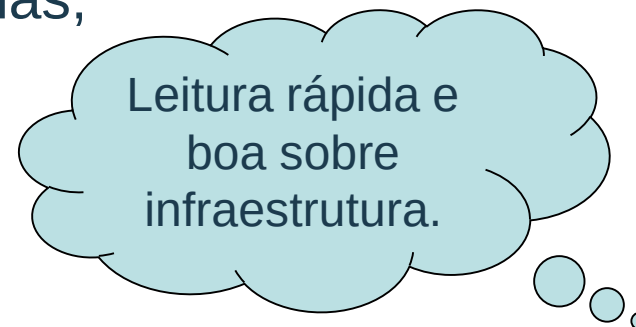


Projeto de interfaces e projeto de componentes

- Os projeto de interfaces, segundo Pressman (2006), representa como as informações entram e saem de um sistema e como essas informações trafegam entre as estruturas desse sistema, definidas no projeto arquitetural.
- O projeto de componentes se refere à ligação dos elementos de uma infraestrutura de tecnologia da informação.

O projeto de interface pode ser dividido em três níveis:

- interfaces internas entre os componentes do sistema;
- interfaces externas entre o sistema e outros sistemas;
- interface com o usuário final.



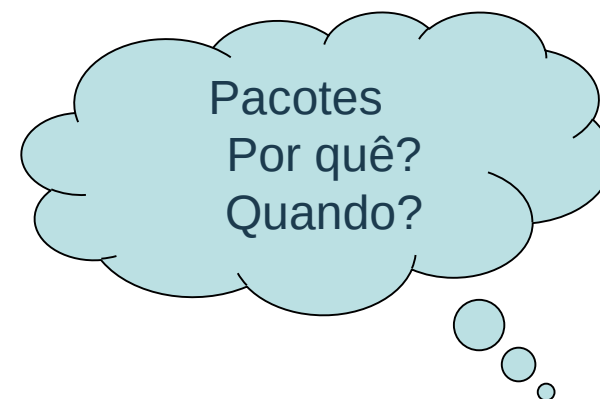
Saiba mais:

ORACLE Corporation Visão geral do Oracle Waveset 8.1.1. Entendendo a arquitetura HA recomendada.

Fonte: <https://docs.oracle.com/cd/E19225-01/821-0762/ghzqp/index.html>, 2010. Consultado em 25/06/2020.

Especificando as interfaces dos objetos

- Na arquitetura em camadas, os objetos são organizados em blocos e a comunicação entre essas camadas se dá por um protocolo definido por objetos de negócio que contêm as informações trafegadas entre essas camadas.
- O protocolo de comunicação entre as camadas de negócio, apresentação e integração, define uma interface de comunicação entre esses objetos e essas camadas.
- A representação desse modelo é feita com o diagrama de pacotes da UML.



Saiba mais:

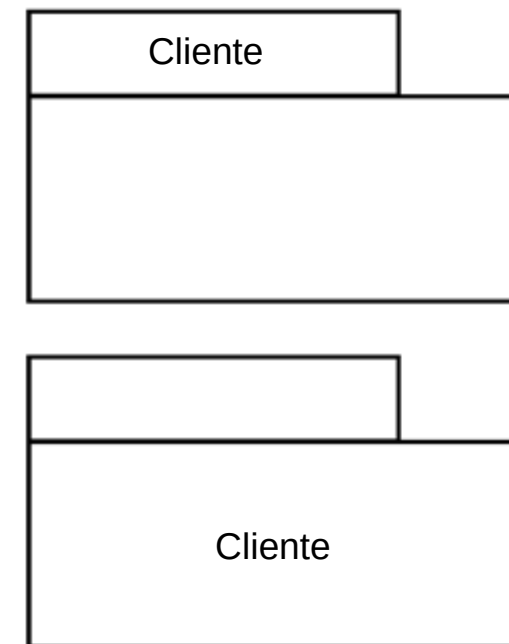
IBM Knowledge Center. Especificando Diagramas UML Padrão para Pacotes.

Fonte:

https://www.ibm.com/support/knowledgecenter/pt-br/SS4JE2_7.5.5/com.ibm.xtools.modeler.doc/topics/tsetdefdiag.html. Consultado em 25/06/2020.

Diagrama de pacotes

- O pacote é utilizado com o propósito de agrupar logicamente objetos, entendendo-se um objeto como qualquer elemento passível de agrupamento no processo de modelagem do sistema: classes, objetos, componentes e casos de uso (LARMAN, 2007).
- A função principal do pacote é organizar objetos que possuem características semelhantes, em agrupamentos maiores e que podem ser facilmente interpretados.
- A figura ao lado mostra os tipos de representação de pacotes e o quadro abaixo, sua aplicação.



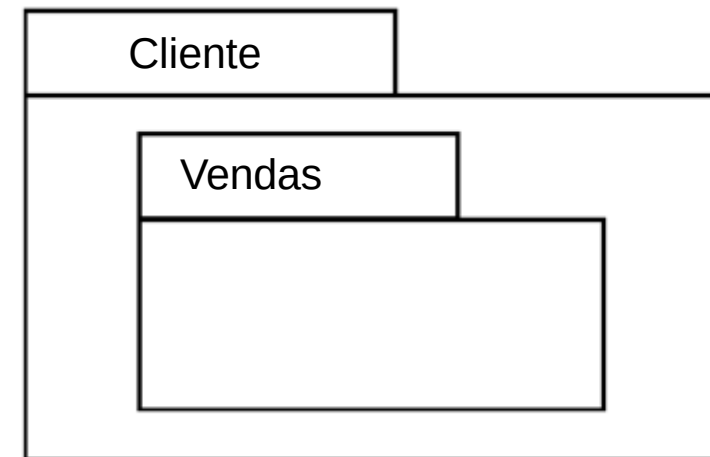
Fonte: VERSOLATTO (2015).

As duas representações podem ser usadas em qualquer instante, porém, pode ser adotado o seguinte padrão:

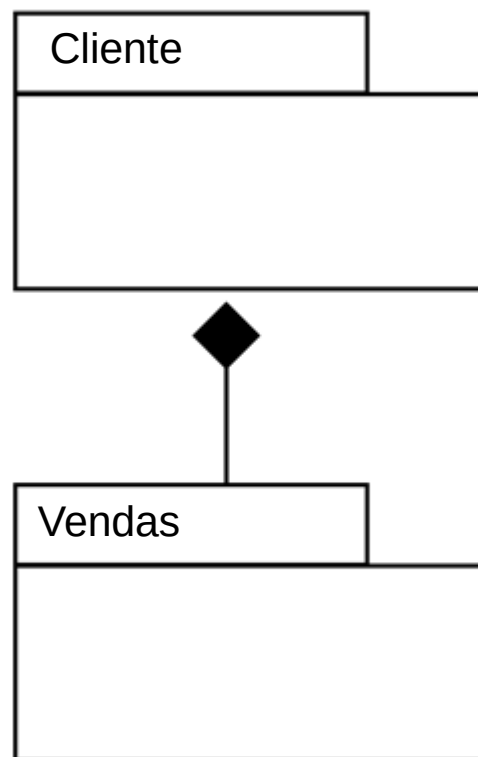
- nome Cliente na guia do pacote (figura de cima) é usada para apresentar os objetos que o compõem;
- nome Cliente no bloco do pacote (figura de baixo) é usado para identificar um determinado pacote.

Diagrama de pacotes – Composição

- Pacote é um agrupamento de elementos e pode utilizar também um pacote como agrupamento de outros pacotes; no caso, um pacote está contido em outro, como mostra a figura.
- O pacote “Vendas” contido no pacote “Cliente” é chamado de subpacote.
- Notação: “Cliente::Vendas”.



Fonte: VERSOLATTO (2015).

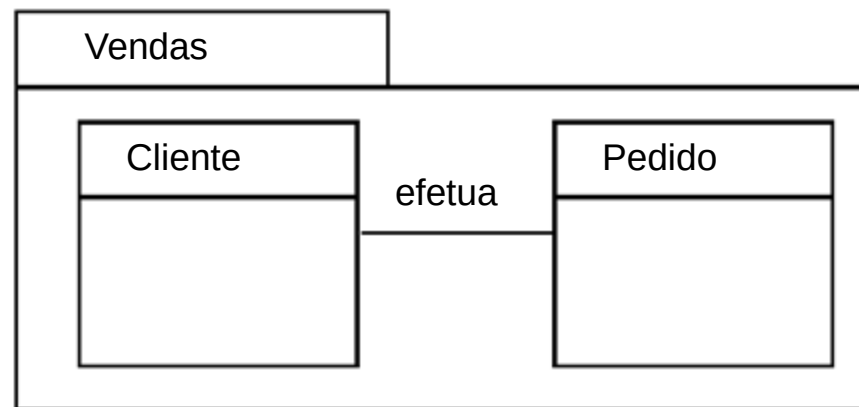


- A relação de pacotes pode ser representada por uma relação de composição, da mesma forma que é representada a composição de classes, como mostra a figura.
- No caso, o pacote “Cliente” é composto pelo pacote “Vendas”.

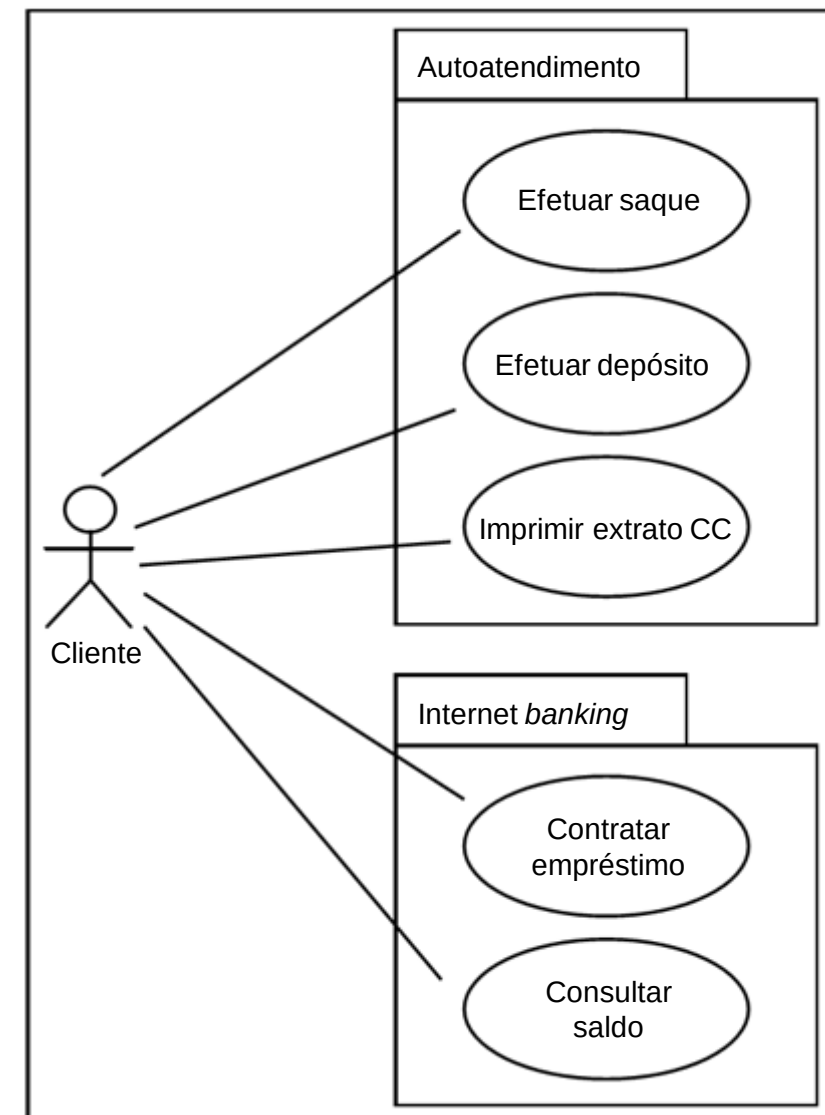
Fonte: VERSOLATTO (2015).

Diagrama de pacotes – Casos de uso e classes

- Um pacote pode conter também um grupo de casos de uso (figura à direita) ou um grupo de classes (figura abaixo).
- O pacote como agrupamento de casos de uso mostra dois canais de execução com casos específicos para cada pacote.
- O pacote como um agrupamento de classes recebe a notação: “Vendas::Cliente” e “Vendas::Pedido”.



Fonte: VERSOLATTO (2015).



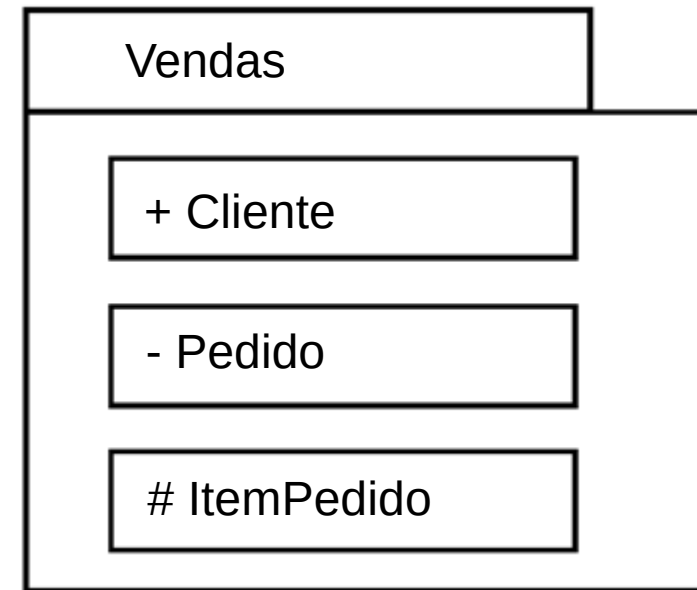
Fonte: VERSOLATTO (2015).

Diagrama de pacotes – Visibilidade dos elementos

- A visibilidade dos elementos segue o mesmo conceito de visibilidade de atributos e métodos.

Temos três níveis de visibilidade:

- Público: representado pelo sinal de positivo (+).
- Privado: representado pelo sinal de negativo (-).
- Protegido: representado pelo sinal sustenido (#).



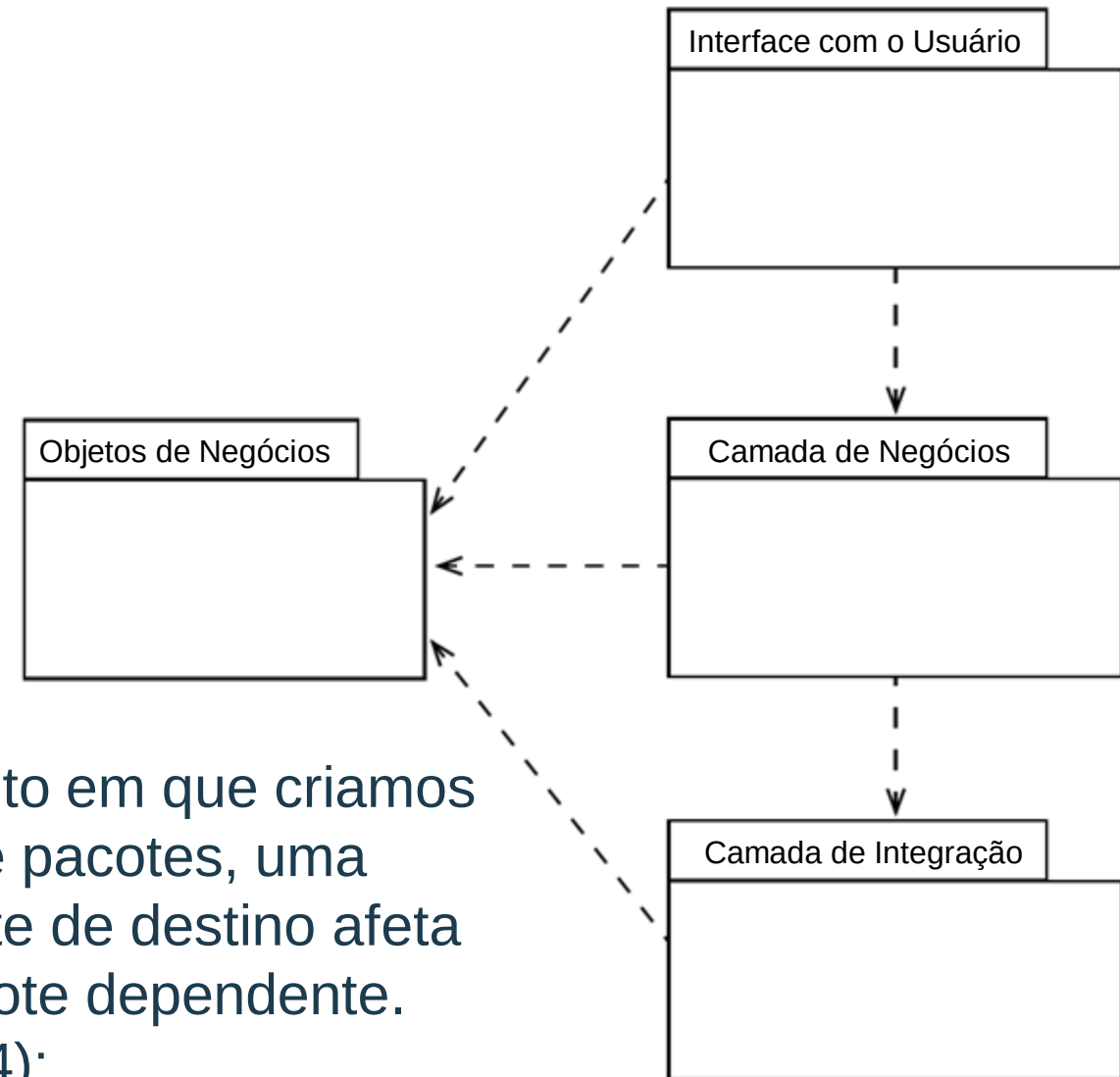
Fonte: VERSOLATTO (2015).

Análise do diagrama de pacotes da figura:

- Elemento “+ Cliente” é público, logo, é visível por todos outros pacotes.
- Elemento “ - Pedido” é privado, logo, só é visível pelos elementos do mesmo pacote.
- Elemento “# ItemPedido”, logo, só é visível em pacotes-filhos (relação de herança) ao qual pertence.

Diagrama de pacotes – Dependência simples

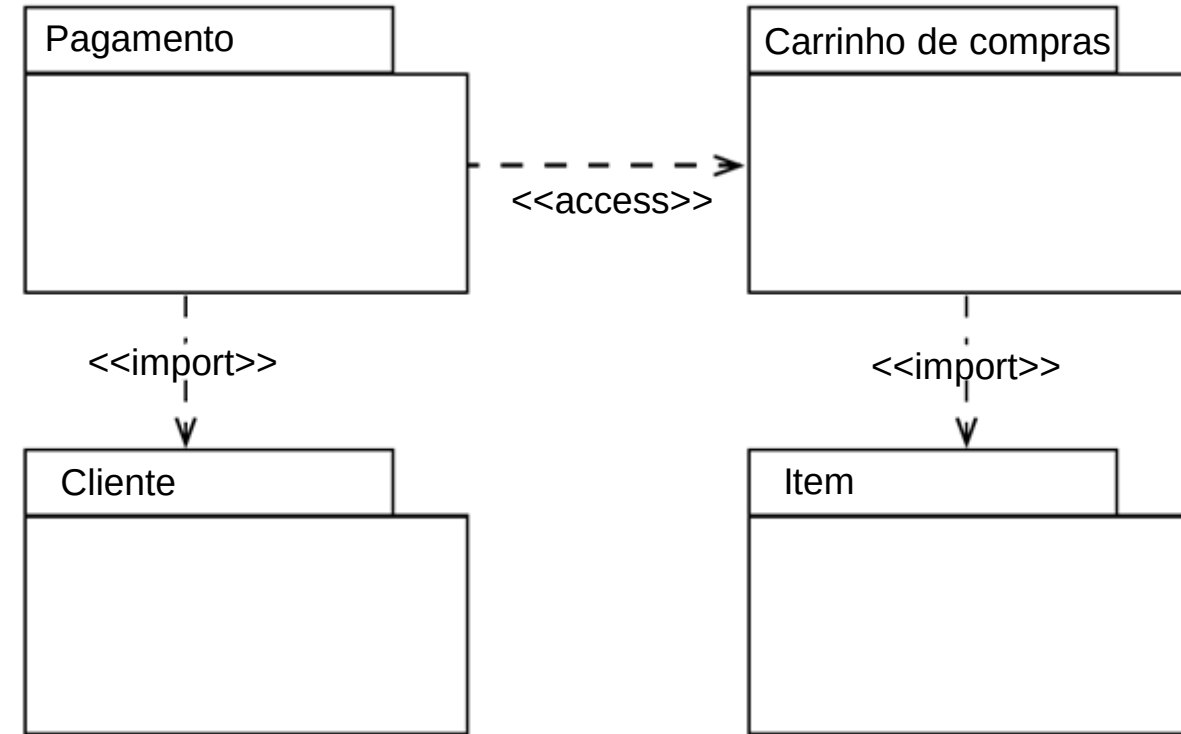
- Dependência simples é quando um elemento de um pacote possui alguma relação com um elemento de outro pacote.
- As classes dos pacotes “Interface com o Usuário”, “Camada de Negócios” e “Camada de Integração” possuem ligação com a classe “Objetos de Negócios”.
- No caso, existe uma dependência dos outros pacotes em relação ao pacote “Objetos de Negócios”.
 - A partir do momento em que criamos dependência entre pacotes, uma alteração no pacote de destino afeta diretamente o pacote dependente. (MEDEIROS, 2004):



Fonte: VERSOLATTO (2015).

Diagrama de pacotes – Dependência estereotipada

- O modelo apresentado na figura possui dois tipos de dependência estereotipada de pacotes:
- Dependência com estereótipo de acesso
`<<access>>`: o pacote dependente é incorporado aos elementos públicos do pacote de destino.
- Dependência com estereótipo de importação
`<<import>>`: os elementos públicos do pacote de destino são adicionados ao pacote dependente.



Fonte: VERSOLATTO (2015).

Saiba mais:

IBM Knowledge Center. Estereótipos UML

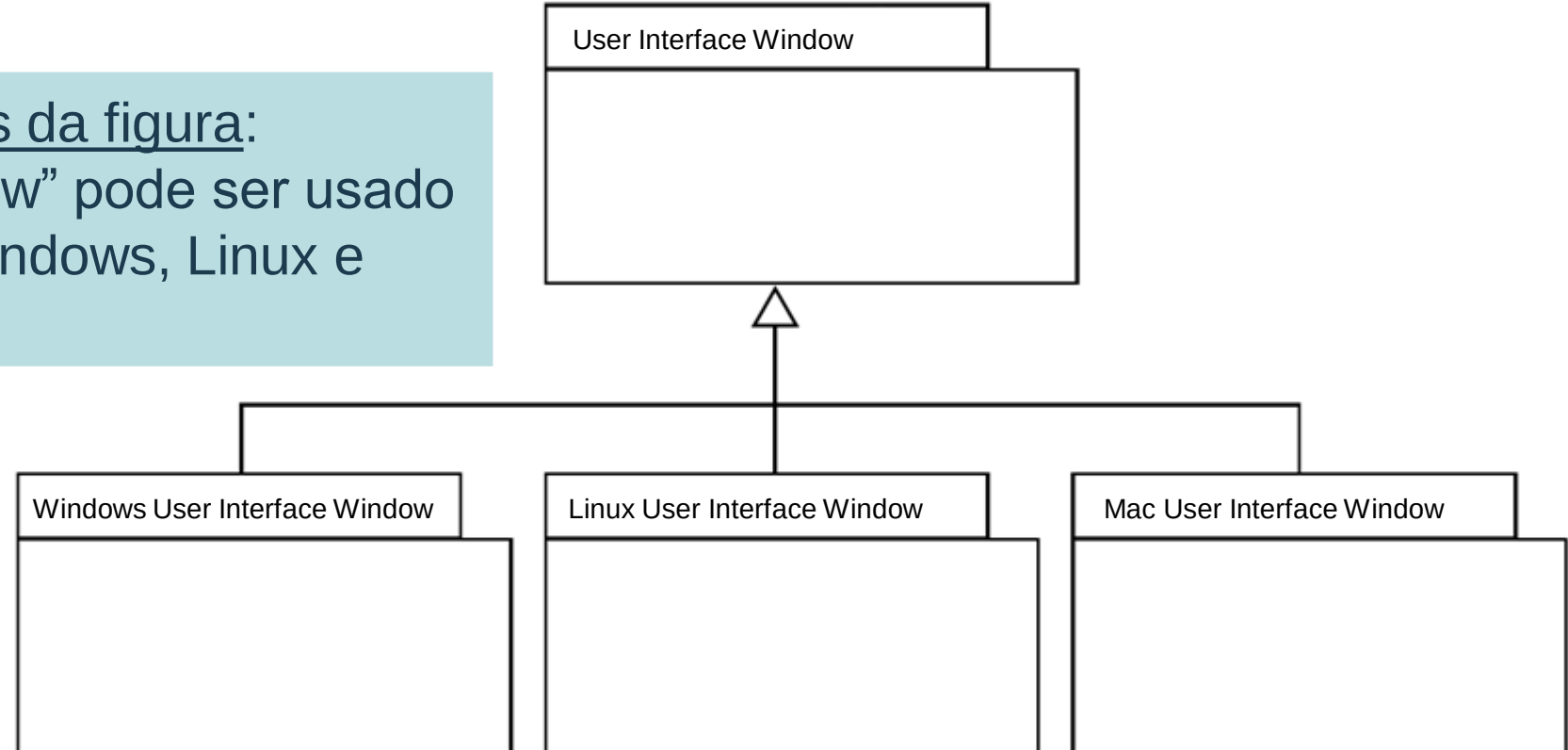
Fonte:

https://www.ibm.com/support/knowledgecenter/pt-br/SS4JE2_7.5.5/com.ibm.xtools.modeler.doc/topics/cstereotyp.html. Consultado em 25/06/2020.

Diagrama de pacotes – Herança de pacotes

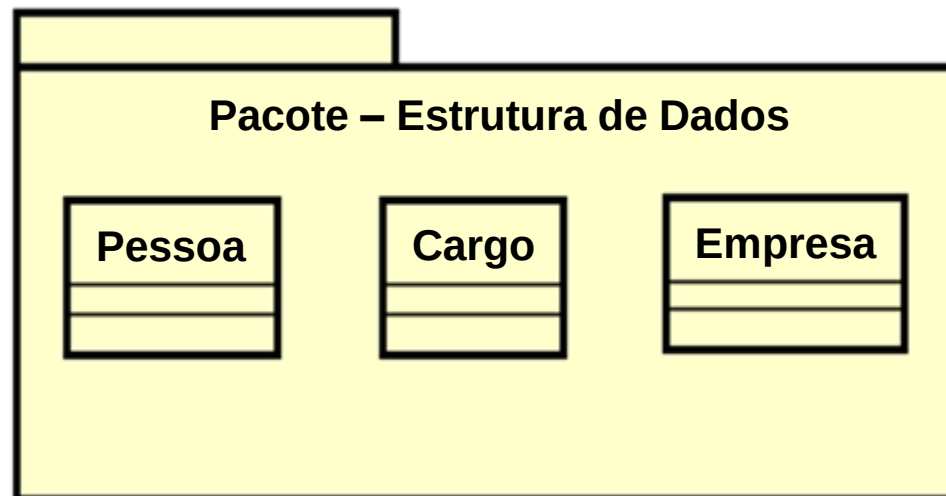
- Herança aplicada a pacotes é utilizada quando desejamos representar uma generalização ou especialização de famílias de pacotes.
- O exemplo da figura mostra pacotes com classes específicas para desenho de formulários de sistemas *desktop* ou baseados em janelas.

Análise do diagrama de pacotes da figura:
O pacote “User Interface Window” pode ser usado pelos sistemas operacionais Windows, Linux e Mac.



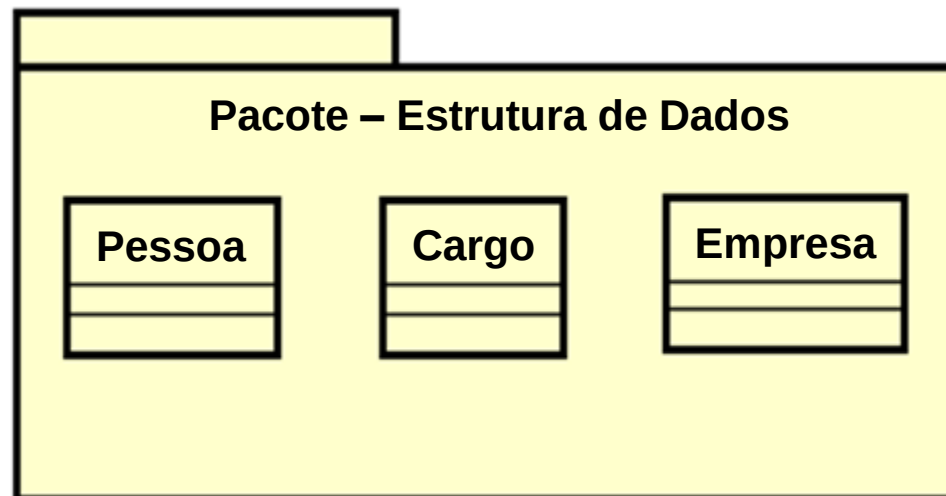
Interatividade

- O pacote Pacote–Estrutura de Dados, na figura abaixo, compõe uma estrutura para um banco de dados de um servidor representado pelo pacote SGBD. Estes pacotes no projeto são agrupados em um único pacote DBMS_Server, que vai servir de orientação para o setor de implantação. Avalie o texto e assinale a alternativa que melhor expressa essa situação.
- a) Este modelo deve ser representado apenas pelos seus atributos e métodos.
 - b) Este modelo não é de um pacote porque está representando um conjunto de classes.
 - c) Isto é uma associação de generalização e não funciona porque SGBD é um gerenciador.
 - d) Isto não é possível porque quando se agrupam pacotes mestres, não podem ser reagrupados.
 - e) O texto está totalmente correto.



Resposta

- O pacote Pacote–Estrutura de Dados, na figura abaixo, compõe uma estrutura para um banco de dados de um servidor representado pelo pacote SGBD. Estes pacotes no projeto são agrupados em um único pacote DBMS_Server, que vai servir de orientação para o setor de implantação. Avalie o texto e assinale a alternativa que melhor expressa essa situação.
- a) Este modelo deve ser representado apenas pelos seus atributos e métodos.
 - b) Este modelo não é de um pacote porque está representando um conjunto de classes.
 - c) Isto é uma associação de generalização e não funciona porque SGBD é um gerenciador.
 - d) Isto não é possível porque quando se agrupam pacotes mestres, não podem ser reagrupados.
 - e) O texto está totalmente correto.



Introdução ao projeto de interfaces com o usuário

- O projeto de interfaces com o usuário está ligado diretamente ao fator usabilidade.
- O projeto de interface do usuário está ligado à engenharia da usabilidade. Não se trata apenas de desenhar a interface do usuário.
- De acordo com a ISO 25010 (2011a), a usabilidade se encaixa entre seis características que delimitam a qualidade de um produto, que são: confiabilidade, funcionalidade, eficiência, manutenibilidade, portabilidade e usabilidade.

Saiba mais:

GOLDEN, E.; JOHN, B. E.; BASS, L. The value of a usability-supporting architectural pattern in software architecture design: a controlled experiment. In: INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING – ICSE, 2005, St. Louis. Proceedings... St. Louis: ICSE, 2005.

Projeto de componentes – Componentização e reúso de *software*

Os cinco princípios básicos de um componente, segundo Cheesman e Daniels (2000), são:

- Um componente obrigatoriamente deve possuir uma especificação.
 - Um componente obrigatoriamente deve possuir uma implementação.
 - Um componente obrigatoriamente deve seguir uma padronização.
 - Um componente obrigatoriamente deve ter a capacidade de ser empacotado em módulos.
 - Um componente obrigatoriamente deve ter a capacidade de ser distribuído.
 - “O componente é a parte modular, possível de ser implantada e substituível de um sistema, que encapsula a implementação e exhibe o conjunto de interfaces”.
-
- A componentização é o processo de criação de ativos digitais com a finalidade de reutilização em projetos de *software* e sistemas.

Projeto de componentes – Definindo as interfaces externas

- As interfaces externas ao componente ou ao sistema de *software* são determinadas pelos protocolos de comunicação ou contrato de comunicação entre objetos consumidores e provedores de serviço.
- Essa relação de consumo de serviços cria uma dependência entre os componentes. As dependências podem ser classificadas como simples ou estereotipadas.

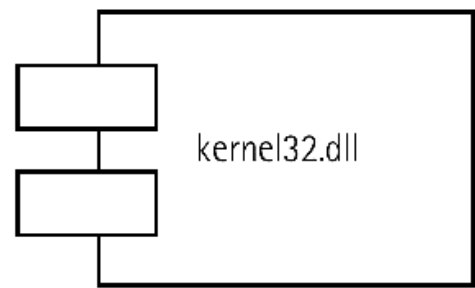
Dependência entre os componentes

Dependências simples são dependências entre componentes delimitadas pela interface, por exemplo, uma dependência entre um executável e uma DLL (*Dynamic-link library*).

Dependências estereotipadas são dependências particulares de um determinado cenário, por exemplo, dependências entre páginas de um *website*, que podem ser marcadas como `<<hiperlink>>`.

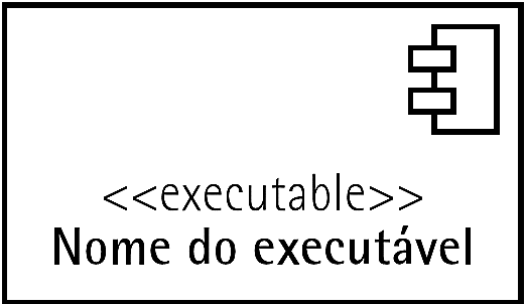
Projeto de componentes – Diagrama de componentes

- A representação de um componente é feita com uma caixa e dois tabs, e o nome do componente, como mostra a figura.



Fonte: VERSOLATTO (2015).

A estereotipação identifica o tipo de componente, como mostra a figura.

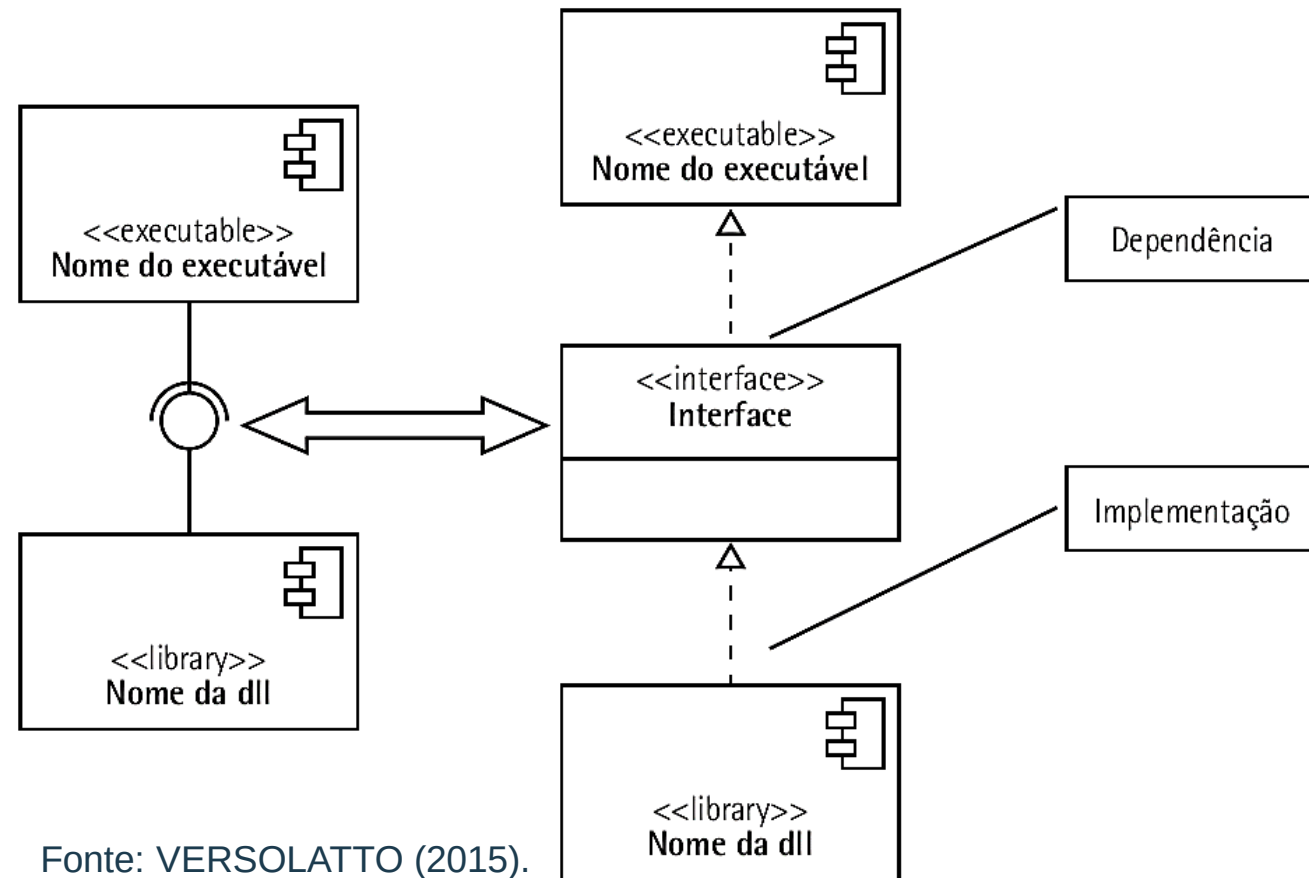


Fonte: VERSOLATTO (2015).

Estereótipo	Tipo de componente
<<executable>>	Componente que pode ser executado.
<<library>>	Biblioteca, que pode ser estática ou dinâmica.
<<database>>	Banco de dados.
<<table>>	Tabela de um banco de dados.
<<document>>	Arquivo.

Diagrama de componentes – Conexão simples

- A representação de interface de componentes é feita como mostra a figura.
- Nessa figura, o componente DLL possui uma interface que é consumida pelo executável; o conector convexo indica a interface <<library>>, e o conector côncavo indica o consumidor <<executable>>.

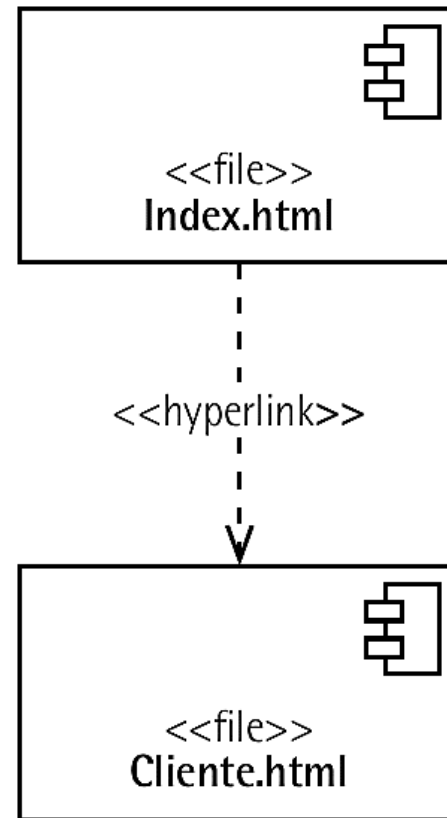


Fonte: VERSOLATTO (2015).

O tipo de dependência representada é de dependências simples entre componentes porque a conexão é feita a partir de interfaces.

Diagrama de componentes – Dependência estereotipada

- Este tipo de dependência é chamada de dependência estereotipada.



Fonte: VERSOLATTO (2015).

Análise sobre o diagrama de componentes:

O diagrama de componentes representa uma visão mais física do sistema de *software*; as partes desse sistema são apresentadas sob uma perspectiva mais concreta, ou seja, podemos empacotar esses componentes e distribuí-los.

Contudo, esse diagrama não nos dá a visão da organização desses componentes sob o enfoque da organização da estrutura física sobre a qual o *software* será implantado e executado.

Como representar, modelar e especificar essas questões importantes será apresentado a seguir no diagrama de distribuição ou de implantação.

Diagrama de distribuição I

- O diagrama de distribuição ou de implantação mostra como os componentes são configurados para execução em “nós” de processamento (LARMAN, 2007).
- Um “nó” de processamento é um recurso computacional que permite a execução de um sistema ou *software*, ou de parte dele, como um componente.
- O “nó” pode ser um computador, um dispositivo móvel ou até mesmo uma estrutura de memória ou um dispositivo periférico.
- O diagrama de implantação é representado por um cubo com sombreado à direita que, por sua vez, representa um “nó” identificado por um nome.

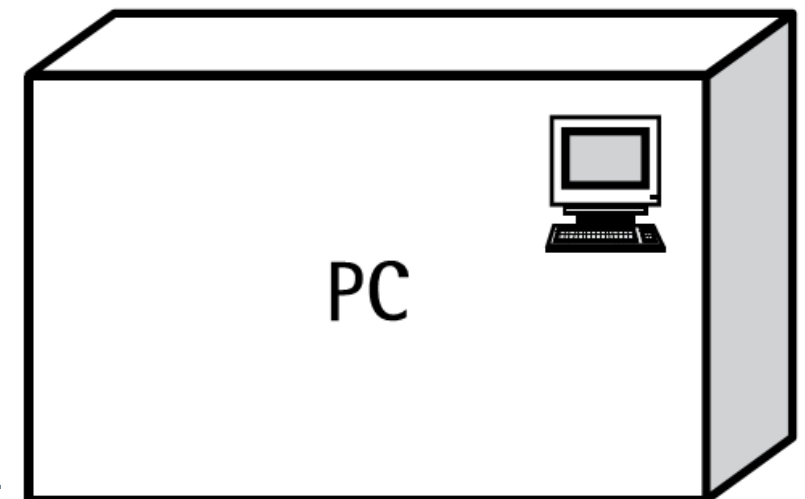
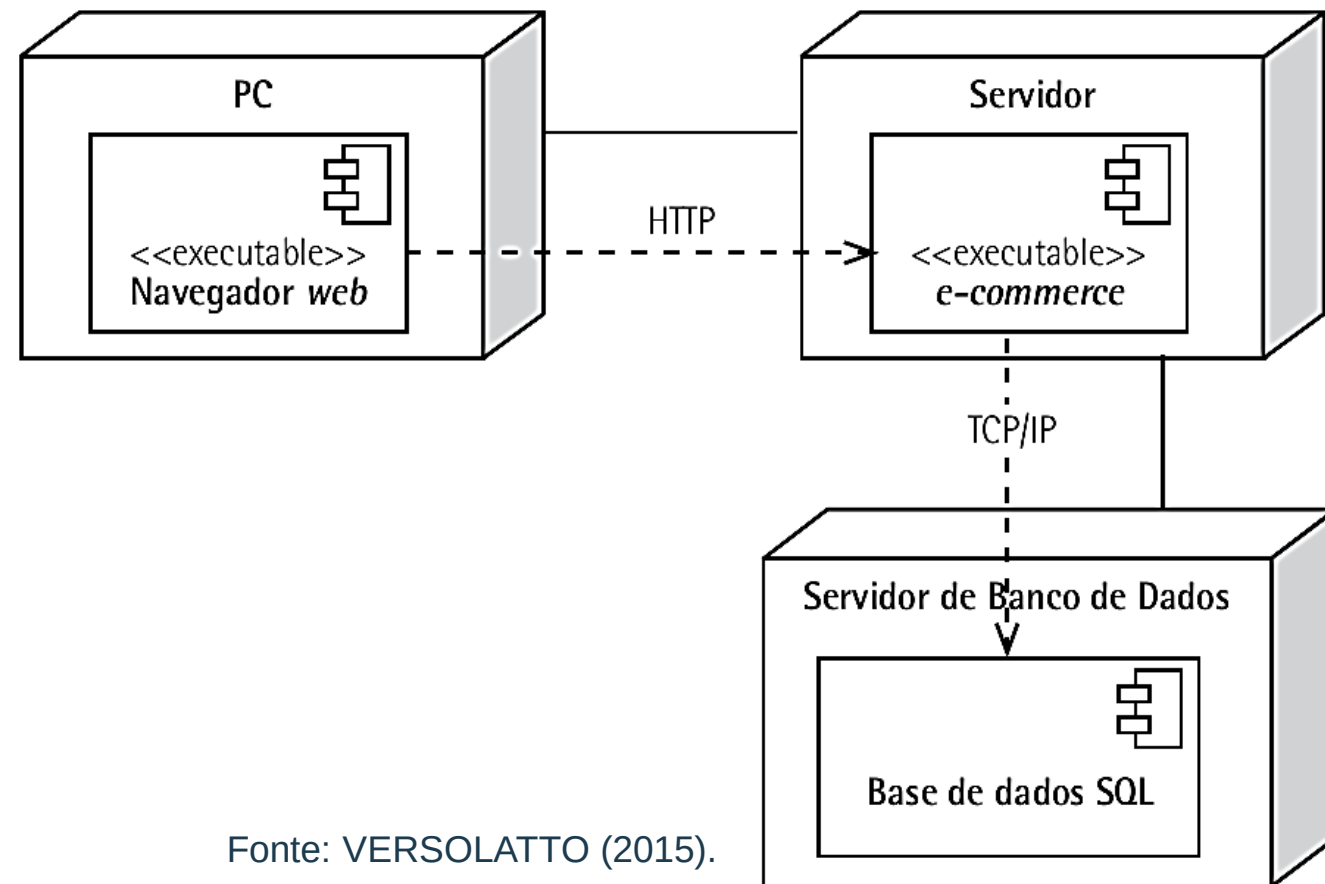


Diagrama de distribuição II

- O diagrama de implantação, permite visualizar as dependências entre os “nós” e como se dá a comunicação entre esses “nós”.
- Por exemplo, em um sistema cliente-servidor, teremos dois “nós”: o cliente e o servidor.

Análise sobre o diagrama de distribuição:
Entre os nós PC e Servidor é feita uma comunicação via protocolo HTTP. O que indica que o acesso do PC ao servidor pode ser feita via browser (navegador).



Fonte: VERSOLATTO (2015).

Interatividade

Analise cada definição como Verdadeira (V) ou Falsa (F) e assinale a alternativa correta.

- I. A componentização pode ser feita com uma visão orientada a objetos.
- II. Um componente deve ter a capacidade de ser distribuído.
- III. Um componente deve ter baixo acoplamento e alta coesão.

- a) F, F, F.
- b) F, V, F.
- c) V, F, V.
- d) V, V, F.
- e) V, V, V.



Resposta

Analise cada definição como Verdadeira (V) ou Falsa (F) e assinale a alternativa correta.

- I. A componentização pode ser feita com uma visão orientada a objetos.
- II. Um componente deve ter a capacidade de ser distribuído.
- III. Um componente deve ter baixo acoplamento e alta coesão.

- a) F, F, F.
- b) F, V, F.
- c) V, F, V.
- d) V, V, F.
- e) V, V, V.

Referências

- CHEESMAN, J.; DANIELS, J. *UML components: a simple process for specifying component based software*. Addison-Wesley, 2000.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION (ISO). *ISO 25010: systems and software engineering – Systems and software quality requirements and evaluation (square) – system and software quality models*. Geneve: ISO, 2011a.
- LARMAN, C. *Utilizando UML e padrões: uma introdução à análise e aos projeto orientados a objetos e ao processo unificado*. 2. ed. Porto Alegre: Bookman, 2007.
- UML-DIAGRAMS. *Timing diagrams*, 2009-2014a. Disponível em:<<http://www.uml-diagrams.org/timing-diagrams.html>>. Acesso em: 28 abr. 2015.

ATÉ A PRÓXIMA!